Lab 4 – Part 1: Introduction to ROS2

M. J. Khojasteh

Fall 2026

1 Objective

Lab 4 Part 1 introduces the fundamental Command Line Interface (CLI) tools in the **Robot Operating System 2 (ROS2)**. In Lab 1, you worked with ROS1; in this part, you will practice the corresponding ROS2 workflows and observe differences in how the two systems operate. We will use the **ROS2 Humble** distribution on **Ubuntu 22.04** for this lab.

This guide follows the official ROS2 Humble tutorials. Rather than reproducing every tutorial, it links to the relevant modules and provides instructions about what to focus on.

Connection to Part 2: UR5e Robot Arm

This lab is divided into two parts. In **Part 1**, you will learn the ROS2 tools and communication concepts needed to work with robotic systems. In **Part 2**, you will apply this foundation to the **UR5e collaborative robot arm**, using ROS2 with **MoveIt** to visualize the robot, plan motions, and execute joint trajectories in simulation and on the real robot. Understanding nodes, topics, packages, and launch files in Part 1 will help you follow how the UR5e driver, controllers, and motion-planning tools work together in Part 2.

Upon completion of Part 1, students should be able to:

1. Identify and explore ROS2 packages and executables.
2. Run ROS2 nodes from the terminal.
3. List, inspect (echo), and publish messages to ROS2 topics.
4. Create a custom ROS2 message and use it in a publisher/subscriber pair.
5. Visualize and explain node communication using `rqt_graph`.

2 Introduction

2.1 Linux/Ubuntu and ROS2

ROS2 is commonly developed and tested on Linux. For this lab, use the ROS2 **Humble** distribution with Ubuntu **22.04**, the combination used on our lab machines. Other platforms and installations exist, but commands in this guide assume this environment. If you use your own computer, make sure that ROS2 Humble and the required tutorial packages are installed.

2.2 ROS2

2.2.1 What is ROS?

The **Robot Operating System (ROS)** is an open-source robotics software framework and collection of tools, rather than a traditional operating system. It supports communication between independent programs (*nodes*), hardware integration, reusable packages, and common robotics development workflows. Nodes exchange typed messages using topics; ROS also supports services and actions for other communication patterns.

2.2.2 How and Why ROS is Used

Robotic systems often contain independent programs for sensors, motion control, perception, and planning. ROS helps these components communicate without requiring every component to be part of one large program. A *publisher* sends messages on a named *topic*, and a *subscriber* receives messages from that topic when the message types and communication settings are compatible. Packages organize nodes, message definitions, configuration files, and other resources.

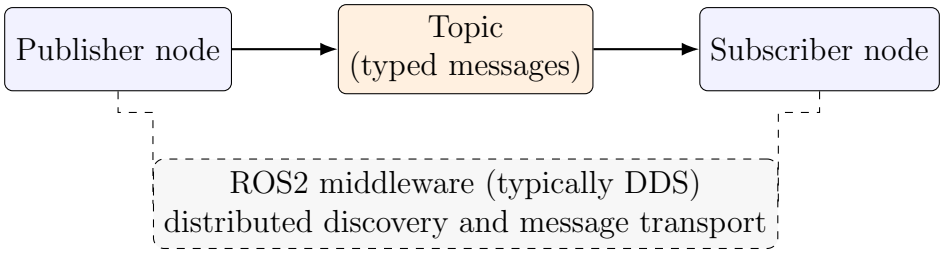


Figure 1: Simplified ROS2 publish/subscribe communication model.

ROS1 vs. ROS2: In ROS1, nodes register through a central `roscore`/ROS Master. ROS2 does **not** require `roscore`: nodes typically discover one another using the underlying DDS-based middleware. In both versions, publishers and subscribers communicate through topics using compatible message types. ROS2 uses commands such as `ros2 node list` and `ros2 topic echo`, rather than the ROS1 commands `rosnode list` and `rostopic echo`.

3 ROS2 Tutorials

3.1 Beginner: CLI Tools

3.1.1 Configuring Environment

1. Please follow the tutorial for Configuring Environment
2. The most important thing to note from this is that you MUST either source your workspace for every terminal/tab you open, or add the command to your shell startup script.
3. When we source our workspace, we are telling ROS where to find relevant packages.

3.1.2 Understanding Nodes

1. Please follow the tutorial for Understanding Nodes
2. Nodes are a fundamental part of ROS, make sure you understand what they are/how they work. If you don't ask your TA.

3.1.3 Understanding Topics

1. Please follow the tutorial for Understanding Topics
2. Topics are also fundamental, make sure you understand what they are/how they work. If you don't ask your TA.
3. rqt_graph is another powerful tool in your ROS arsenal, make sure to use this if you're ever confused how everything connects.

3.1.4 Understanding Services

1. Please follow the tutorial for Understanding Services
2. Services are fundamental to ROS2, but won't be used as much in the scope of this course. Ensure you understand what they are/how they work, but you won't be tasked with implementing them.

3.1.5 Understanding Parameters

1. Please follow the tutorial for Understanding Parameters
2. Parameters are fundamental to ROS2, but won't be used as much in the scope of this course. Ensure you understand what they are/how they work, but you won't be tasked with implementing them.

3.1.6 Understanding Actions

1. Please follow the tutorial for Understanding Actions
2. Actions are fundamental to ROS2, but won't be used as much in the scope of this course. Ensure you understand what they are/how they work, but you won't be tasked with implementing them.

3.1.7 Using rqt_console to view logs

1. Please follow the tutorial for Using rqt_console to view logs
2. If you ever want to see error or warning messages in more detail, this is useful.

3.1.8 Launching Nodes

1. Please follow the tutorial for Launching Nodes
2. Launch files are a very useful and versatile tool. They act as a centralized space where you can modify every aspect of code that runs when you launch a node.
3. We will use launch files often in conjuncture with our own python scripts to achieve our own desired behavior.

3.1.9 Recording and Playing Back Data

1. Please follow the tutorial for Recording and Playing Back Data
2. If you have ever worked with log files before the idea is the same.
3. We write the published/subscribed messages from a node to a big file so we can analyze it later for debugging.

3.2 Beginner: Client Libraries

3.2.1 Using colcon to Build Packages

1. Please follow the tutorial for Using colcon to Build Packages
2. We'll be using the colcon commands often, you may want to write some of the key ones down.
3. If you have never worked with Linux/programming in an intensive fashion such as this please ask your TA to explain it to you.

3.2.2 Creating a Workspace

1. Please follow the tutorial for Creating a Workspace
2. Your workspace is where you'll put all programs you write/import. If you are using a lab computer skip the repo cloning part (Maybe), else it is good practice to run through this.

3.2.3 Creating a Package

1. Please follow the tutorial for Creating a Package
2. A package is just a way for you to organize your code if it were to be shared with others. The scope of this class won't have you writing any packages, but you should know what they are as you'll be downloading and using packages in later labs.

3.2.4 Writing a Simple Publisher and Subscriber (Python)

1. Please follow the tutorial for Writing a Simple Publisher and Subscriber
2. These are the basis of what we will be working with in ROS in the future. Make sure you understand the code and the theory thoroughly. If you need help or explanations ask your TA.

4 Deliverables

Complete the following five deliverables using **ROS2**.

1. Create a simple publisher and subscriber loop that publishes increasing numbers using either Python or C++.
2. Create a custom ROS2 message with at least two fields.
3. Create a publisher and subscriber node pair that uses your custom message.
4. Produce an image of the node network using `rqt_graph`. The image should show your publisher, subscriber, and the topic connecting them.
5. Explain how a publisher/subscriber network works in ROS2 and what your `rqt_graph` means.

5 Report

Make sure to submit a tech memo which follows the sekeleton/guidelines that are located in MyCourses.

You only need to submit ONE report for both part 1 and part 2 of this lab. Your report should include the following:

1. All deliverables.
2. Answers to all questions asked within the lab document.
3. A summary of what was accomplished and learned within the lab.